PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Domain 2

Prepared by: Fantastic 5

Team Members:

1. Bibianne Zheyee Joseph
2. Hang Xiu Jun
3. Kishanea A/P Jeyakumar
4. Liew Jia Xin
5. Ng Guan Yik

Table of Contents

1. Project Overview 3

2. Background & Business Objective 3

3. Product Purpose 5

4. System Functionalities 5

5. User Stories & Use Cases 10

6. Features Included (Scope Definition) 10

7. Features Not Included (Scope Control) 12

8. Assumptions & Constraints 13

9. Risks & Questions Throughout Development 13

1. Project Overview

Field	Details
Project Name	FinSight AI
Version	1.0
Domain	AI for Economic Empowerment & Decision Intelligence
Problem Statement	Individuals struggle to make informed purchase decisions due to limited financial awareness and lack of personalised decision tools.
Target Domain	Personal finance decision intelligence for young Malaysian adults.
Proposed Solution	A hybrid purchase decision assistant that combines a deterministic financial engine with AI-powered explanations using Z.AI ILMU-GLM-5.1. that analyses a user's financial situation and recommends whether to Buy, Delay, or Avoid a purchase, along with peer benchmarking and scenario simulation.

AI Model Architecture

FinSight AI is powered by the **Z.AI Model (Z.AI ILMU-GLM-5.1)**, which serves as the core reasoning engine for financial decision-making.

Processing Flow:

- **Inputs**
  - Monthly income, expenses, savings
  - Purchase goal (e.g., laptop, travel)
  - Time horizon and constraints

- **Z.AI Model Reasoning**
  - Evaluates affordability and sustainability

    - Detects financial risk (e.g., negative cash flow, low savings buffer)
    - Simulates future financial impact over time
    - Performs trade-off analysis between current spending and future goals

- **Outputs**
    - Decision: *Proceed / Delay / Not advisable*
    - Risk level (Low / Medium / High)
    - Financial impact breakdown
    - Actionable suggestions

2. Background & Business Objective

- **Background of the Problem:** Many students, young professionals, SME owners & gig workers make impulsive purchase decisions without understanding the true financial impact. With rising costs of living and stagnant entry-level wages, a single poor financial decision (e.g., buying a phone on credit when savings are insufficient) can cascade into months of financial stress. Current solutions such as general budgeting apps focus on tracking past spending rather than advising on future decisions. They do not provide contextual, AI-driven reasoning about whether a specific purchase makes sense given the user's current financial situation.

- **Importance of Solving This Issue:** A real-time, AI-powered purchase advisor closes the gap between financial data and financial wisdom. Rather than just showing numbers, FinSight AI explains consequences, compares the user to their peers, and suggests alternatives, further empowering better decisions before money is spent.

- **Strategic Fit / Impact:** FinSight AI directly addresses the hackathon’s Domain 2 requirement: AI for Economic Empowerment & Decision Intelligence. The solution demonstrates Z.AI ILMU-GLM-5.1 as the core reasoning engine, such that if the GLM is removed, the system can no longer generate explanations, alternatives, or personalized recommendations.

3. Product Purpose

3.1. Main Goal of the System

To provide users with an AI-powered purchase decision assistant that analyses their financial situation in real time and delivers clear, personalized guidance (Buy, Delay, or Avoid), backed by financial simulation, peer benchmarking, and GLM reasoning.

3.2. Intended Users (Target Audience)

User Type	Description	How System Adapts
Students	Fixed monthly allowance, limited financial literacy	Typically lower income → scoring engine flags affordability risk earlier
Gig Workers	Irregular income	Variable income reflected in inputs → simulation highlights recovery uncertainty
Young Professionals	Growing income but managing loans and commitments	EMI + expenses affect score and recovery time
SME Owners	Variable income, managing business and personal finances	Higher volatility reflected through income/expense inputs

4. System Functionalities

4.1. Description

FinSight AI operates as a modular financial intelligence platform. The user inputs their financial profile and a purchase item. The system runs deterministic financial simulations, scores the decision, retrieves peer benchmarks from a cleaned dataset, and sends structured context to the

Z.AI GLM for AI-driven reasoning and recommendations. The system follows a hybrid architecture where deterministic modules handle all financial computations, while the LLM is strictly used for reasoning and explanation. This ensures consistency, reduces hallucination risk, and keeps financial calculations transparent and verifiable.

4.2. Key Functionalities:

Feature	Module	Description
Purchase Input System	app.py (frontend)	User enters item name, price, income, expenses, savings, optional EMI, urgency level, and category
Financial Simulation Engine	simulator.py	Calculates net disposable income, recovery months, risk level, and 4 savings trajectories (Buy Now, Delay at 90%, Skip, Cheaper Alt at 70%)
Decision Scoring Engine	scoring.py	Produces a 0-100 financial health score based on surplus ratio, savings depth, recovery time, urgency, and loan status
Peer Benchmarking	benchmarks.py	Compares user's income, expenses, and savings against Asian peers from a cleaned 6,551-row financial dataset
AI Recommendation Engine	ai_explainer.py	Z.AI ILMU-GLM-5.1 receives financial metrics and generates decision summary, trade-off analysis, explanation with user's actual numbers, 2-3 alternatives, and one specific action

Feature	Module	Description
Scenario Comparison Chart	app.py (frontend)	Visual graph showing savings trajectory across all 4 scenarios over 12 months
Receipt Scanner	ai_explainer.py	Vision-based image upload to auto-extract item name and price from a receipt or price tag
Follow-up Q&A	ai_explainer.py	Multi-turn conversational follow-up questions answered by GLM with full financial context

4.3. AI Model & Prompt Design

4.3.1. Model Selection

Model Used: Z.AI ILMU-GLM-5.1 via the Anthropic-compatible API endpoint (https://api.ilmu.ai/anthropic).

Justification: ILMU-GLM-5.1 was selected because it supports structured JSON output for consistent response parsing, handles multi-turn conversational context for follow-up Q&A, is accessible through the Anthropic-compatible API enabling seamless integration with Claude Code tooling, and demonstrates strong contextual reasoning, essential for explaining financial trade-offs in plain language with Malaysian market context.

4.3.2. Prompting Strategy

FinSight AI uses a structured single-turn prompt strategy with a persistent system prompt.

System Prompt (persistent): Establishes the GLM as a Malaysian personal finance advisor. Instructs it to reference local platforms and provide concise, data-driven advice using the user's actual numbers.

User Prompt (dynamic per request): Injects a compact financial context block (~300 tokens) containing user profile, purchase details, simulation results, peer benchmarks, and the rule-based decision. Requests a structured JSON response with 6 fields: summary, trade-off, explanation, alternatives, action, and confidence.

Rationale: Single-turn structured prompting was chosen over multi-step agentic prompting to minimise latency and token cost per request. The deterministic financial engine pre-computes all metrics before the GLM call, so the GLM focuses solely on reasoning and language generation — not calculation.

4.3.3. Context & Input Handling

Parameter	Value	Handling
Max tokens per request	2,048 tokens	Set in API payload; prevents runaway responses
Prompt size (approx.)	~300 tokens	Kept compact by pre-computing metrics in simulator.py before sending to GLM
Oversized inputs	Not applicable	User inputs are numeric fields — no free-text injection risk
Temperature	0.3	Low temperature ensures consistent, factual financial advice

Parameter	Value	Handling
Cache TTL	120 seconds	Identical inputs skip the API call entirely using MD5-keyed in-memory cache

4.3.4. Fallback & Failure Behavior

Error Type	Detection	Fallback Behaviour
API key missing	__NO_KEY__ prefix	Shows offline message; displays rule-based reason and category-specific alternatives
HTTP 429/502/503/504	HTTPError code check	Retries up to 2 times with exponential backoff (1s, 2s); shows “server busy” message
Timeout (>90s)	URLError / TimeoutError	Shows timeout message; prompts user to retry
Malformed JSON response	_parse_json_response()	3-stage fallback: clean parse, truncation repair, regex per-field extraction
GLM hallucination / off-topic	Structured JSON enforcement	Required keys enforced; missing fields default to empty string — UI hides blank sections

5. User Stories & Use Cases

As a...	I want to...	So that...
Students	enter my monthly allowance and a purchase price	I know if I can afford it without running out of money
Gig Workers	understand financial impact of irregular income	I avoid unstable commitments
Young Professionals	compare alternatives before buying	I optimize spending decisions
Any user	compare Buy Now vs Delay vs Skip scenarios visually	I can see the long-term impact of my decision
Any user	upload a receipt image	The app auto-fills the item price without manual typing
Any user	ask follow-up questions after getting a recommendation	I can clarify the advice before making a decision

<u>Use Cases (Main Interactions)</u>

Use Case 1: Purchase Decision Analysis

- User fills in financial profile (income, expenses, savings, optional EMI)
- User enters item name, price, category, and urgency
- System validates inputs via validate_inputs()

- simulator.py computes metrics and 4 scenario trajectories
- scoring.py generates a 0-100 financial health score
- benchmarks.py retrieves peer comparison from dataset
- ai_explainer.py sends structured prompt to ILMU-GLM-5.1
- GLM returns JSON with summary, trade-off, explanation, alternatives, action
- UI displays decision badge, metrics, AI advice, and scenario chart

Use Case 2: Receipt Scanning

- User uploads a receipt or price tag image
- ai_explainer.scan_receipt_image() sends image to GLM vision
- GLM extracts item name and price from image
- App auto-populates the purchase input fields

Use Case 3: Follow-up Q&A

- After receiving a recommendation, user types a follow-up question
- ai_explainer.get_followup_answer() sends full chat history + financial context to GLM
- GLM responds in plain English with contextual awareness of the user's situation

6. Features Included (Scope Definition)

Feature	Status
Purchase input form with financial profile	Included
Financial simulation engine (4 scenarios, EMI support, edge case handling)	Included
Decision scoring engine (0-100 score with risk classification)	Included
Peer benchmarking from cleaned Asian financial dataset (6,551 rows)	Included
AI recommendation via Z.AI ILMU-GLM-5.1 (structured JSON output)	Included
Scenario comparison chart (12-month savings trajectory)	Included
Receipt / price tag image scanner (GLM vision)	Included
Multi-turn follow-up Q&A with GLM	Included
In-memory response caching (120s TTL)	Included
Input validation with user-friendly error messages	Included

7. Features Not Included (Scope Control)

Feature	Reason
User accounts / login system	Out of scope for hackathon MVP — session state used instead
Persistent data storage / database	Pandas in-memory only; no SQL or cloud DB
Real-time product price scraping	Alternative prices are AI-suggested, not scraped from live marketplaces
Mobile app (iOS / Android)	Web-only via Streamlit; mobile deployment not in scope

Feature	Reason
Multi-currency support	RM only; USD dataset converted to RM at fixed rate (4.7)
Investment or loan planning features	Decision intelligence focuses on purchase decisions only
Production deployment / cloud hosting	Local Streamlit run only for hackathon

8. Assumptions & Constraints

8.1 LLM Cost Constraint

Estimated token cost per session: approximately 400-600 tokens per analysis request (300 token prompt + up to 300 token response). At ILMU-GLM-5.1 pricing, this is minimal per session.

- Cost management decisions made: (1) Pre-compute all financial metrics in simulator.py before the GLM call — the GLM only does reasoning, not arithmetic. (2) In-memory LRU cache with 120-second TTL skips duplicate API calls for identical inputs. (3) Temperature set to 0.3 to avoid verbose or repetitive outputs. (4) max_tokens capped at 2,048 to prevent runaway responses.

8.2 Technical Constraints

Constraint	Detail
API dependency	System requires ILMU_API_KEY environment variable to be set for GLM features to work
Dataset coverage	Peer benchmarks use a synthetic Asia-region dataset (6,551 rows)
USD to RM conversion	Fixed rate of 4.7 used; not real-time exchange rate

Constraint	Detail
Image scan support	Receipt scanner depends on GLM vision capability; falls back gracefully if unsupported
Simulation assumptions	Income and expenses are assumed constant throughout the 12-month simulation period
No negative balance modelling	Savings balance is NOT clamped to zero — debt accumulation is modelled realistically

8.3 User Input Constraints

- Income must be greater than zero
- All monetary values must be non-negative
- Purchase price must be greater than zero
- Monthly EMI cannot exceed available monthly surplus
- If expenses exceed 200% of income, user is prompted to check their values

9. Risks & Questions Throughout Development

Risk	Impact	Mitigation
GLM API timeout	HIGH — core feature unavailable	60-90s timeout, 2 retries with backoff, graceful fallback to rule-based reason
GLM returns malformed JSON	MEDIUM — display breaks	3-stage JSON parser with regex fallback; missing fields default to empty string
Synthetic dataset not representative of Malaysian users	MEDIUM — benchmarks may be inaccurate	Clearly labelled as 'peer benchmarks (Asia region)'; framed as reference, not ground truth
API key exposure	HIGH — financial and security risk	Key stored in environment variable only; never hardcoded in source code
GLM gives inconsistent advice across sessions	MEDIUM — erodes user trust	Temperature = 0.3 for deterministic outputs; structured JSON enforces response format
User inputs unrealistic values	LOW — bad recommendations	validate_inputs() catches edge cases; UI shows warning messages before analysis runs